PYTHON ACTIVITY-BASED LEARNING

10 Selected Case Studies — Complete Practical Record

Student Details

Name	Chitturi Charanya Sri
Roll Number	26001454
Branch	CSD
Section	2
Subject	Python Programming

This document includes the problem statement, objectives, assumptions, algorithm, flowchart, pseudocode, Python source code, test cases, sample output captures, and conclusion for each selected case study.

Selected from the user's supplied Python Activity-Based Learning case-study list. Rates and thresholds are educational assumptions where stated.

Case Study 1: Progressive Electricity Billing

1. Problem Statement

• Write a Python program to calculate an electricity bill from the number of units consumed using progressive unit slabs and a fixed charge.

2. Objectives

• Accept electricity units from the user.

• Apply different rates to different consumption slabs.

• Add a fixed charge and display the final bill.

3. Assumptions

• Units consumed cannot be negative.

• First 100 units cost ■2 per unit.

• Units 101–200 cost ■3 per unit; units above 200 cost ■5 per unit.

• A fixed charge of ■50 is added.

4. Algorithm

1. Start the program.

2. Read the number of units consumed.

3. If units are 100 or less, charge ■2 per unit.

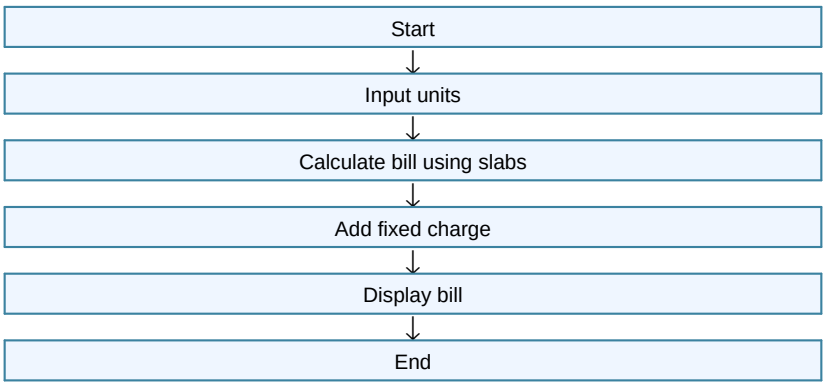
4. If units are 101–200, charge ■2 for the first 100 units and ■3 for the remaining units.

5. Otherwise, charge ■2 for the first 100, ■3 for the next 100, and ■5 for the remaining units.

6. Add the fixed charge of ■50.

7. Display the bill and stop.

5. Flowchart



6. Pseudocode

```
START
INPUT units
IF units <= 100
    bill = units * 2
ELSE IF units <= 200
    bill = 100*2 + (units-100)*3
ELSE
    bill = 100*2 + 100*3 + (units-200)*5
END IF
bill = bill + 50
OUTPUT bill
STOP
```

7. Python Source Code

```
units = int(input("Enter electricity units: "))

if units < 0:
```

```
    print("Invalid units")
elif units <= 100:
    bill = units * 2
elif units <= 200:
    bill = 100 * 2 + (units - 100) * 3
else:
    bill = 100 * 2 + 100 * 3 + (units - 200) * 5

if units >= 0:
    bill = bill + 50
    print("Electricity Bill =", bill)
```

8. Test Cases

Input / Scenario	Expected Result
Units = 150	Bill = ■400
Units = 80	Bill = ■210
Units = -5	Invalid units

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Enter electricity units: 150
Electricity Bill = 400
```

10. Conclusion

The program calculates an electricity bill by applying progressive rates and a fixed charge. It demonstrates input, arithmetic operators, and conditional statements.

Case Study 2: Hospital Billing and Patient Classification

1. Problem Statement

• Write a program to calculate a patient's hospital bill from consultation and treatment fees, then classify the patient by age.

2. Objectives

• Read patient age, consultation fee, and treatment fee.

• Calculate the total bill.

• Apply a 10% concession to patients aged 60 or above and display the category.

3. Assumptions

• Age and fees are non-negative integers.

• Patients aged 60 or above receive a 10% discount.

• Patients younger than 60 are classified as Regular.

4. Algorithm

1. Start.

2. Input age, consultation fee, and treatment fee.

3. Add the consultation and treatment fees.

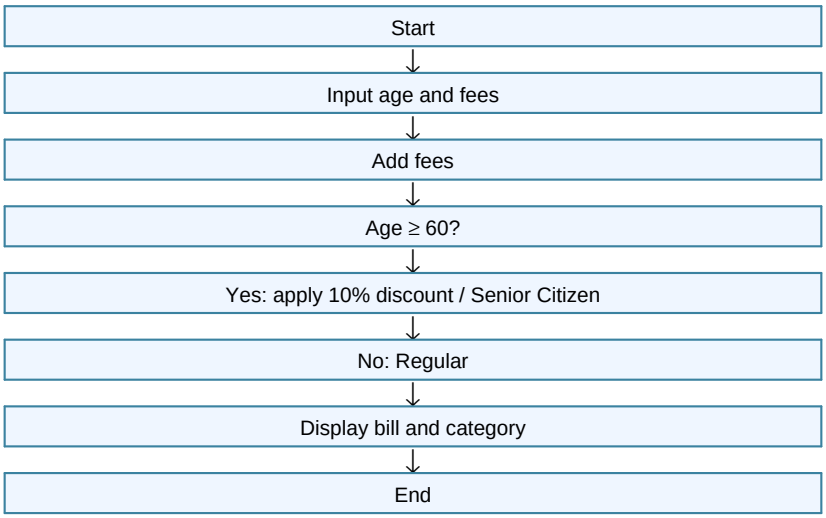
4. If age is 60 or above, reduce the bill by 10% and classify as Senior Citizen.

5. Otherwise, classify as Regular.

6. Display the final bill and category.

7. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT age, consultation, treatment
bill = consultation + treatment
IF age >= 60
    bill = bill * 0.90
    category = "Senior Citizen"
ELSE
    category = "Regular"
END IF
OUTPUT bill, category
STOP
```

7. Python Source Code

```
age = int(input("Enter patient age: "))
```

```
consultation = int(input("Enter consultation fee: "))
treatment = int(input("Enter treatment fee: "))

if age < 0 or consultation < 0 or treatment < 0:
    print("Invalid input")
else:
    bill = consultation + treatment

    if age >= 60:
        bill = bill * 0.90
        category = "Senior Citizen"
    else:
        category = "Regular"

    print("Final Hospital Bill =", bill)
    print("Patient Category:", category)
```

8. Test Cases

Input / Scenario	Expected Result
Age 65, fees ■500 and ■1500	Bill = ■1800.0; Senior Citizen
Age 25, fees ■500 and ■1500	Bill = ■2000; Regular
Negative fee	Invalid input

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Enter patient age: 65
Enter consultation fee: 500
Enter treatment fee: 1500
Final Hospital Bill = 1800.0
Patient Category: Senior Citizen
```

10. Conclusion

The program totals hospital charges, applies a conditional concession, and classifies patients based on age.

Case Study 3: Attendance Recovery Computation

1. Problem Statement

• Calculate a student's attendance percentage and, when it is below 75%, find the minimum number of consecutive classes the student must attend to reach 75%.

2. Objectives

• Calculate attendance percentage.

• Check the 75% eligibility threshold.

• Use a loop to find the minimum additional classes required.

3. Assumptions

• Total classes must be greater than zero.

• Attended classes must be between zero and total classes.

• Every future class is attended; the threshold is 75%.

4. Algorithm

1. Start.

2. Input classes attended and total classes.

3. Validate the inputs.

4. Calculate percentage = attended / total × 100.

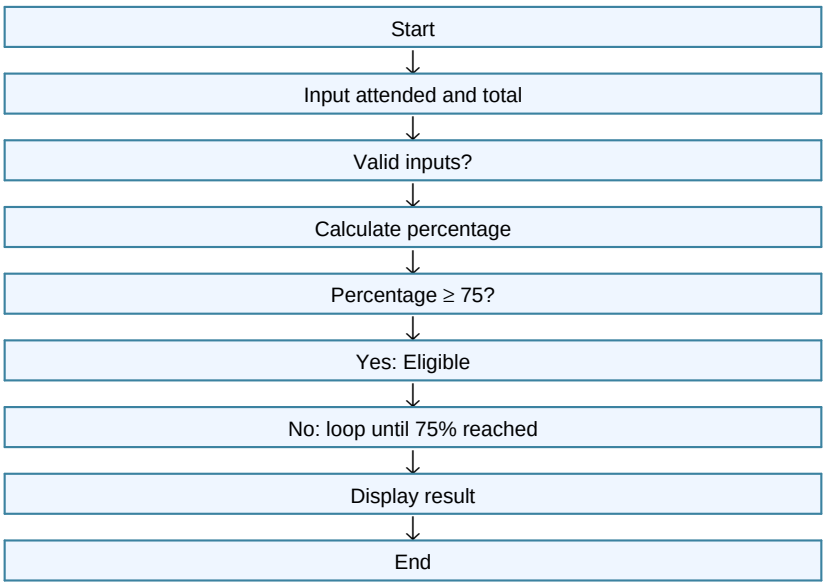
5. If percentage is at least 75, display Eligible.

6. Otherwise, increase an additional-class counter until (attended + counter) / (total + counter) is at least 75%.

7. Display the number of classes required.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT attended, total
IF total <= 0 OR attended < 0 OR attended > total
    OUTPUT "Invalid input"
    STOP
END IF
percentage = attended / total * 100
OUTPUT percentage
IF percentage >= 75
```

```
    OUTPUT "Eligible"
ELSE
    required = 0
    WHILE (attended+required)/(total+required)*100 < 75
        required = required + 1
    END WHILE
    OUTPUT required
END IF
STOP
```

7. Python Source Code

```
attended = int(input("Enter classes attended: "))
total = int(input("Enter total classes: "))

if total <= 0 or attended < 0 or attended > total:
    print("Invalid input")
else:
    percentage = attended / total * 100
    print("Attendance Percentage =", percentage)

    if percentage >= 75:
        print("Eligible for Examination")
    else:
        required = 0
        while (attended + required) / (total + required) * 100 < 75:
            required += 1
        print("Not Eligible")
        print("More classes to attend =", required)
```

8. Test Cases

Input / Scenario	Expected Result
Attended 60, total 100	60.0%; 60 more classes
Attended 80, total 100	80.0%; Eligible
Attended 5, total 0	Invalid input

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Enter classes attended: 60
Enter total classes: 100
Attendance Percentage = 60.0
Not Eligible
More classes to attend = 60
```

10. Conclusion

The program uses percentages, conditions, and a while loop to determine attendance eligibility and the minimum recovery requirement.

Case Study 4: ATM Transaction Sequence Simulation

1. Problem Statement

• Create a menu-driven ATM simulation that supports deposits, withdrawals, balance checks, and exit.

2. Objectives

• Maintain an account balance.

• Accept repeated menu choices.

• Reject non-positive amounts and withdrawals larger than the available balance.

3. Assumptions

• The starting balance is ■1,000.

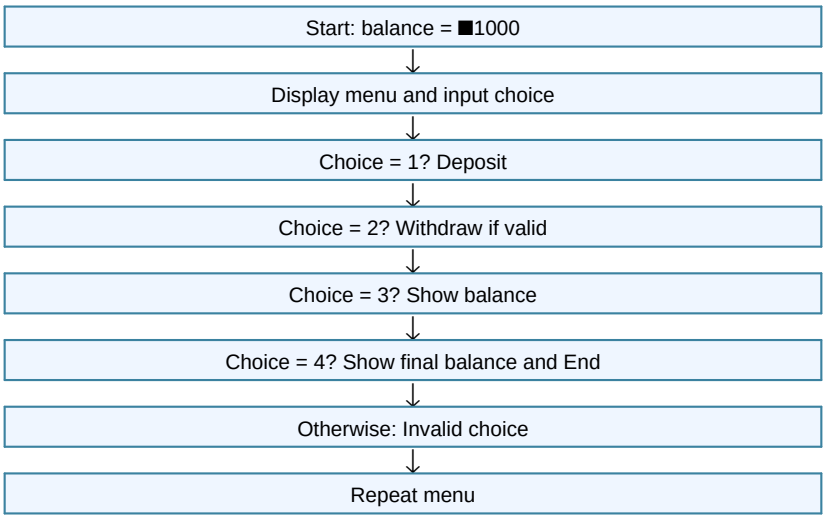
• Transactions are simulated only; no real bank account is accessed.

• Deposits and withdrawals must be positive amounts.

4. Algorithm

1. Start with balance ■1,000.
2. Display the ATM menu.
3. Read the user's choice.
4. For deposit, add a positive amount to balance.
5. For withdrawal, subtract a positive amount only if sufficient balance exists.
6. For balance inquiry, display the balance.
7. For exit, display the final balance and stop.
8. Repeat the menu for other valid operations.

5. Flowchart



6. Pseudocode

```
START
balance = 1000
REPEAT
    DISPLAY menu
    INPUT choice
    IF choice = 1
        INPUT amount
        IF amount > 0: balance = balance + amount
        ELSE OUTPUT "Invalid amount"
    ELSE IF choice = 2
        INPUT amount
        IF amount > 0 AND amount <= balance
            balance = balance - amount
```

```
        ELSE OUTPUT "Invalid amount or insufficient balance"
    ELSE IF choice = 3
        OUTPUT balance
    ELSE IF choice = 4
        OUTPUT balance
        STOP LOOP
    ELSE
        OUTPUT "Invalid choice"
    END IF
UNTIL choice = 4
STOP
```

7. Python Source Code

```
balance = 1000

while True:
    print("\n1. Deposit")
    print("2. Withdraw")
    print("3. Check Balance")
    print("4. Exit")
    choice = int(input("Enter your choice: "))

    if choice == 1:
        amount = int(input("Enter deposit amount: "))
        if amount > 0:
            balance += amount
            print("Deposit Successful")
        else:
            print("Invalid Amount")
    elif choice == 2:
        amount = int(input("Enter withdrawal amount: "))
        if amount > 0 and amount <= balance:
            balance -= amount
            print("Withdrawal Successful")
        else:
            print("Invalid Amount or Insufficient Balance")
    elif choice == 3:
        print("Balance =", balance)
    elif choice == 4:
        print("Final Balance =", balance)
        break
    else:
        print("Invalid Choice")
```

8. Test Cases

Input / Scenario	Expected Result
Deposit ■500, then exit	Final balance ■1500
Withdraw ■200 from ■1000	Balance becomes ■800
Withdraw ■2000 from ■1000	Insufficient balance message

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice: 1
Enter deposit amount: 500
Deposit Successful

Enter your choice: 4
Final Balance = 1500
```

10. Conclusion

The program simulates common ATM operations using a loop, menu selection, conditions, and balance updates.

Case Study 5: Warehouse Inventory Reconciliation

1. Problem Statement

• Calculate the closing stock of each product from opening stock, incoming stock, and dispatched stock, and identify products that need replenishment.

2. Objectives

• Store product names and quantities in lists.

• Calculate closing stock for each product.

• Flag stock below the replenishment threshold.

3. Assumptions

• All stock quantities are non-negative integers.

• Closing stock = opening stock + incoming stock – dispatched stock.

• Replenishment is required when closing stock is below 10.

4. Algorithm

1. Start.

2. Create lists of product names, opening stock, incoming stock, and dispatched stock.

3. Loop through each product.

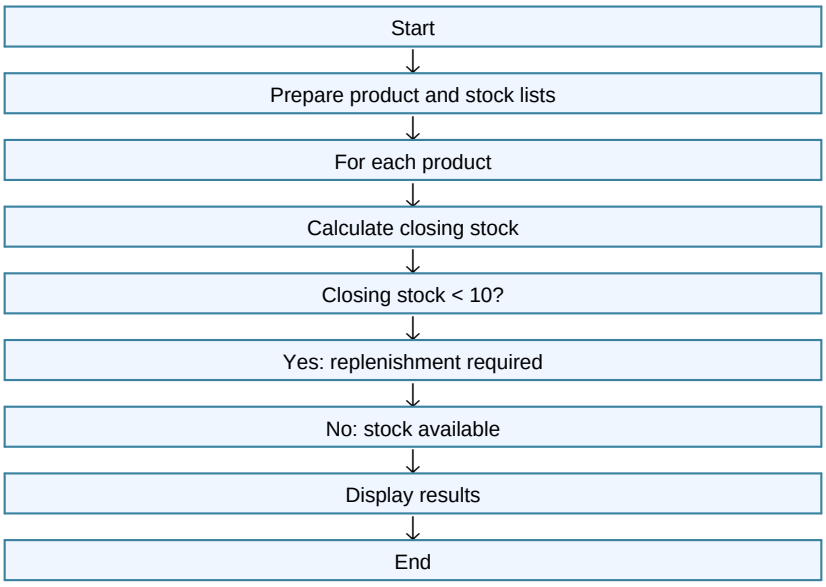
4. Calculate closing stock.

5. Display the closing stock.

6. If closing stock is below 10, display Replenishment Required; otherwise display Stock Available.

7. Stop.

5. Flowchart



6. Pseudocode

```
START
products = ["Rice", "Sugar", "Oil"]
opening = [50, 30, 20]
incoming = [20, 10, 5]
dispatched = [60, 25, 22]
FOR i from 0 to length(products)-1
    closing = opening[i] + incoming[i] - dispatched[i]
    OUTPUT products[i], closing
    IF closing < 10
        OUTPUT "Replenishment Required"
    ELSE
```

```
        OUTPUT "Stock Available"
    END IF
END FOR
STOP
```

7. Python Source Code

```
products = ["Rice", "Sugar", "Oil"]
opening = [50, 30, 20]
incoming = [20, 10, 5]
dispatched = [60, 25, 22]

for i in range(len(products)):
    closing = opening[i] + incoming[i] - dispatched[i]

    print("\nProduct:", products[i])
    print("Closing Stock:", closing)

    if closing < 10:
        print("Replenishment Required")
    else:
        print("Stock Available")
```

8. Test Cases

Input / Scenario	Expected Result
Rice: 50 opening, 20 incoming, 60 dispatched	Closing stock 10; Stock Available
Oil: 20 opening, 5 incoming, 22 dispatched	Closing stock 3; Replenishment Required
Sugar: 30 opening, 10 incoming, 25 dispatched	Closing stock 15; Stock Available

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Product: Rice
Closing Stock: 10
Stock Available

Product: Sugar
Closing Stock: 15
Stock Available

Product: Oil
Closing Stock: 3
Replenishment Required
```

10. Conclusion

The program uses parallel lists and a for loop to reconcile inventory and flag products with low closing stock.

Case Study 6: Student Performance Trend Analysis

1. Problem Statement

• Compare each student's subject marks across two examinations and report whether performance improved, declined, or stayed the same.

2. Objectives

• Store subject names and marks in lists.

• Calculate the difference between second and first exam marks.

• Classify each subject's performance trend.

3. Assumptions

• Marks are from 0 to 100.

• The sample data contains three subjects.

• Difference = second examination marks – first examination marks.

4. Algorithm

1. Start.

2. Create lists of subjects and marks for two examinations.

3. Loop through each subject.

4. Calculate the difference between the two marks.

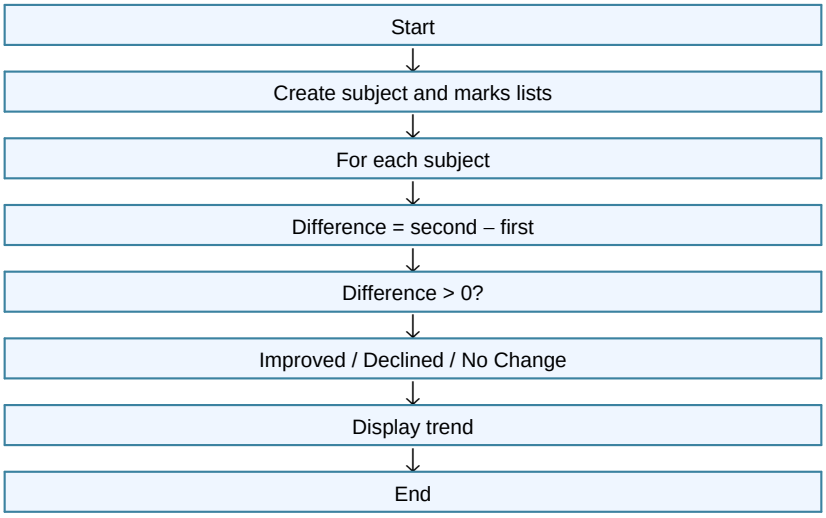
5. If the difference is positive, report improvement.

6. If negative, report decline by its absolute value.

7. Otherwise report no change.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
subjects = ["Python", "Maths", "English"]
exam1 = [60, 75, 80]
exam2 = [70, 65, 85]
FOR i from 0 to length(subjects)-1
    difference = exam2[i] - exam1[i]
    OUTPUT subjects[i]
    IF difference > 0
        OUTPUT "Improved by", difference
    ELSE IF difference < 0
        OUTPUT "Declined by", absolute(difference)
    ELSE
```

```
        OUTPUT "No Change"
    END IF
END FOR
STOP
```

7. Python Source Code

```
subjects = ["Python", "Maths", "English"]
exam1 = [60, 75, 80]
exam2 = [70, 65, 85]

for i in range(len(subjects)):
    difference = exam2[i] - exam1[i]

    print("\nSubject:", subjects[i])
    print("First Exam:", exam1[i])
    print("Second Exam:", exam2[i])

    if difference > 0:
        print("Improved by", difference, "marks")
    elif difference < 0:
        print("Declined by", -difference, "marks")
    else:
        print("No Change")
```

8. Test Cases

Input / Scenario	Expected Result
Python: 60 to 70	Improved by 10 marks
Maths: 75 to 65	Declined by 10 marks
A subject: 80 to 80	No Change

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Subject: Python
First Exam: 60
Second Exam: 70
Improved by 10 marks

Subject: Maths
First Exam: 75
Second Exam: 65
Declined by 10 marks

Subject: English
First Exam: 80
Second Exam: 85
Improved by 5 marks
```

10. Conclusion

The program compares examination marks and classifies trends using lists, arithmetic, loops, and conditional statements.

Case Study 7: Traffic Violation Penalty Calculator

1. Problem Statement

• Calculate a simplified traffic fine based on the offence type and whether it is a repeat offence.

2. Objectives

• Read the offence type and repeat-offence status.

• Select a fine based on the offence.

• Double the practice fine for repeat offences and display a warning.

3. Assumptions

• Practice fine for helmet offence is ■500.

• Practice fine for signal offence is ■1,000.

• Repeat offences double the selected fine.

• These are sample rules for programming practice, not official legal fine amounts.

4. Algorithm

1. Start.

2. Input offence and repeat status.

3. If offence is helmet, set fine to ■500.

4. If offence is signal, set fine to ■1,000.

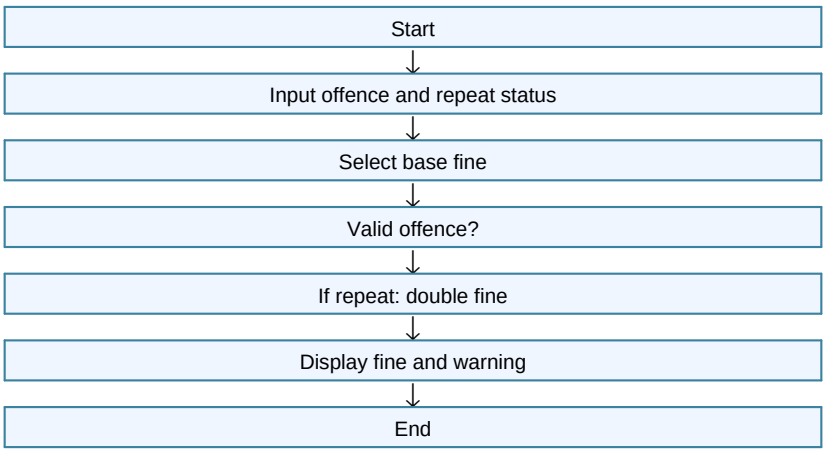
5. Otherwise, display Invalid Offence.

6. If repeat status is yes, double the fine.

7. Display the fine and a repeat-offence warning if applicable.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT offence, repeat
IF offence = "helmet"
    fine = 500
ELSE IF offence = "signal"
    fine = 1000
ELSE
    fine = 0
END IF
IF fine = 0
    OUTPUT "Invalid Offence"
ELSE
    IF repeat = "yes"
        fine = fine * 2
```

```
    END IF
    OUTPUT fine
    IF repeat = "yes"
        OUTPUT "Check for Disqualification"
    END IF
END IF
STOP
```

7. Python Source Code

```
offence = input("Enter offence (helmet/signal): ").lower()
repeat = input("Is it a repeat offence? (yes/no): ").lower()

if offence == "helmet":
    fine = 500
elif offence == "signal":
    fine = 1000
else:
    fine = 0

if fine == 0:
    print("Invalid Offence")
else:
    if repeat == "yes":
        fine = fine * 2

    print("Traffic Fine =", fine)

    if repeat == "yes":
        print("Repeat Offence")
        print("Check for Disqualification")
```

8. Test Cases

Input / Scenario	Expected Result
helmet, no	Fine ■500
signal, yes	Fine ■2000; repeat warning
speeding, no	Invalid Offence

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Enter offence (helmet/signal): helmet
Is it a repeat offence? (yes/no): yes
Traffic Fine = 1000
Repeat Offence
Check for Disqualification
```

10. Conclusion

The program applies simple offence-based rules using string comparison and conditions. Real traffic laws and penalties must be checked from official sources.

Case Study 8: Password Policy Validation

1. Problem Statement

• Validate a sample password by checking its length and whether it contains uppercase letters, lowercase letters, and digits.

2. Objectives

• Check password length.

• Use a loop to inspect each character.

• Report validity and missing requirements.

3. Assumptions

• The minimum password length is eight characters.

• At least one uppercase letter, one lowercase letter, and one digit are required.

• Special characters are optional for this exercise.

4. Algorithm

1. Start.

2. Input the password.

3. Set uppercase, lowercase, and digit flags to False.

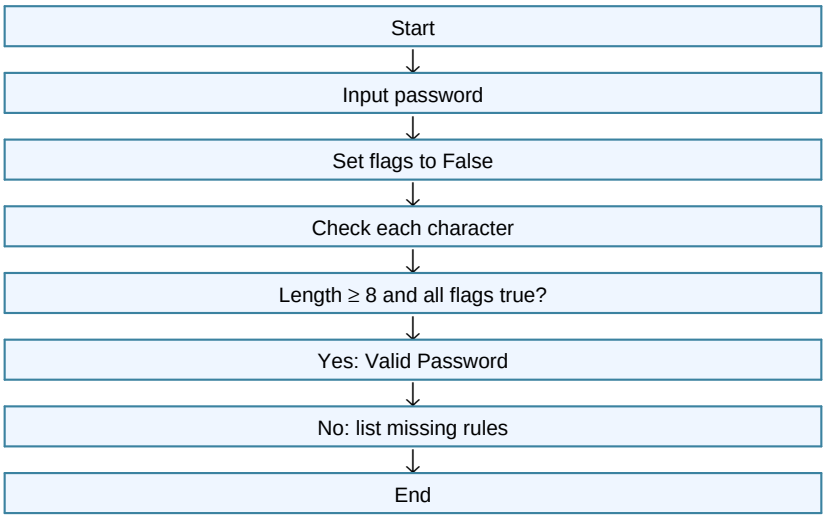
4. Loop through each character and update the appropriate flags.

5. If length and all flags meet the rules, display Valid Password.

6. Otherwise display Invalid Password and each missing requirement.

7. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT password
upper = false
lower = false
digit = false
FOR each character ch in password
    IF ch is uppercase: upper = true
    IF ch is lowercase: lower = true
    IF ch is digit: digit = true
END FOR
IF length(password) >= 8 AND upper AND lower AND digit
    OUTPUT "Valid Password"
ELSE
    OUTPUT "Invalid Password"
    OUTPUT each unmet rule
```

```
END IF
STOP
```

7. Python Source Code

```
password = input("Enter password: ")

upper = False
lower = False
digit = False

for ch in password:
    if ch.isupper():
        upper = True
    elif ch.islower():
        lower = True
    elif ch.isdigit():
        digit = True

if len(password) >= 8 and upper and lower and digit:
    print("Valid Password")
else:
    print("Invalid Password")
    if len(password) < 8:
        print("Password must have 8 characters")
    if not upper:
        print("Add an uppercase letter")
    if not lower:
        print("Add a lowercase letter")
    if not digit:
        print("Add a digit")
```

8. Test Cases

Input / Scenario	Expected Result
Hello123	Valid Password
hello123	Invalid: uppercase missing
Ab1	Invalid: length and lowercase/uppercase/digit checks as applicable

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Enter password: Hello123
Valid Password
```

10. Conclusion

The program demonstrates string methods, loops, Boolean flags, and conditional validation of a password.

Case Study 9: Nested Loop Calendar Generator

1. Problem Statement

• Print a simple monthly calendar using a starting weekday and the number of days in the month.

2. Objectives

• Accept a starting weekday from 0 to 6.

• Align the first date under the correct weekday.

• Print dates in rows of seven days.

3. Assumptions

• Monday is represented by 0 and Sunday by 6.

• The user enters a valid starting weekday from 0 to 6.

• The number of days is between 28 and 31.

• Calendar dates are printed with a simple text layout.

4. Algorithm

1. Start.

2. Input starting weekday and number of days.

3. Validate the values.

4. Print weekday headings.

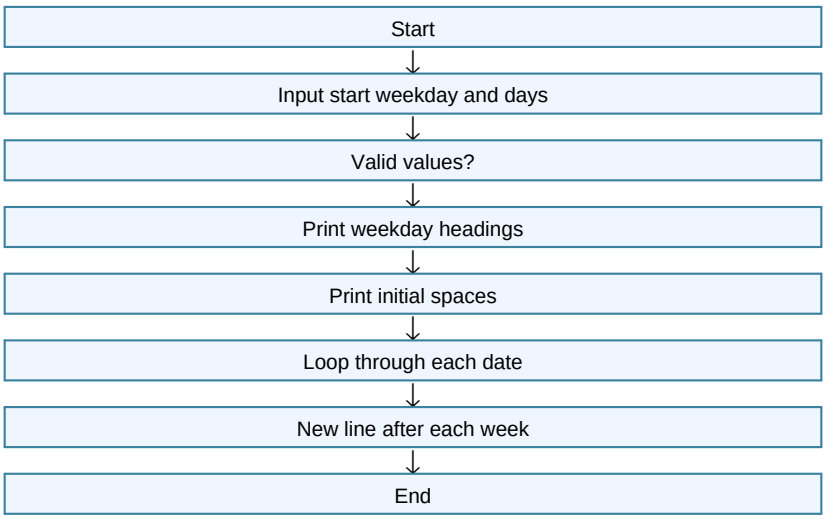
5. Print blank spaces for weekdays before day 1.

6. Loop through dates from 1 to the number of days.

7. Print each date and start a new line after each complete week.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
INPUT start, days
IF start < 0 OR start > 6 OR days < 28 OR days > 31
    OUTPUT "Invalid input"
    STOP
END IF
OUTPUT weekday headings
FOR i from 1 to start
    OUTPUT blank space
END FOR
FOR day from 1 to days
    OUTPUT day in width 3
```

```
    IF (start + day) MOD 7 = 0
        OUTPUT new line
    END IF
END FOR
OUTPUT new line
STOP
```

7. Python Source Code

```
start = int(input("Enter starting weekday (0-6): "))
days = int(input("Enter number of days (28-31): "))

if start < 0 or start > 6 or days < 28 or days > 31:
    print("Invalid input")
else:
    print("Mon Tue Wed Thu Fri Sat Sun")

    for i in range(start):
        print("    ", end="")

    for day in range(1, days + 1):
        print("%3d" % day, end=" ")

        if (start + day) % 7 == 0:
            print()

    print()
```

8. Test Cases

Input / Scenario	Expected Result
Start 0, days 31	Day 1 starts under Monday; 31 dates printed
Start 2, days 30	Day 1 starts under Wednesday; 30 dates printed
Start 7, days 30	Invalid input

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Enter starting weekday (0-6): 2
Enter number of days (28-31): 30
Mon Tue Wed Thu Fri Sat Sun
              1   2   3   4   5
  6   7   8   9  10  11  12
 13  14  15  16  17  18  19
 20  21  22  23  24  25  26
 27  28  29  30
```

10. Conclusion

The program uses loops, conditions, and formatted output to align and print a monthly calendar.

Case Study 10: Food Waste Monitoring System

1. Problem Statement

• Calculate leftover meals and the percentage of food waste for several days, then flag days with wastage above 20%.

2. Objectives

• Store meals prepared and served in lists.

• Calculate leftover meals and waste percentage.

• Identify days where waste exceeds the threshold.

3. Assumptions

• Prepared meals are greater than zero.

• Served meals are between zero and prepared meals.

• Waste percentage = leftover meals / prepared meals × 100.

• Waste above 20% is classified as high.

4. Algorithm

1. Start.

2. Create lists of meals prepared and meals served.

3. Loop through each day.

4. Calculate leftover = prepared – served.

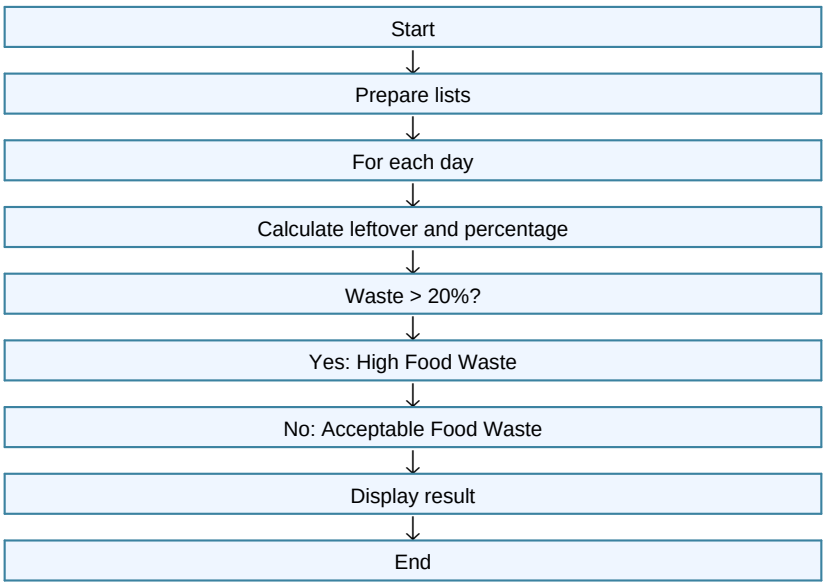
5. Calculate wastage percentage.

6. Display daily figures.

7. If wastage is greater than 20%, display High Food Waste; otherwise display Acceptable Food Waste.

8. Stop.

5. Flowchart



6. Pseudocode

```
START
prepared = [100, 120, 80]
served = [90, 100, 60]
FOR i from 0 to length(prepared)-1
    leftover = prepared[i] - served[i]
    percentage = leftover / prepared[i] * 100
    OUTPUT day, leftover, percentage
    IF percentage > 20
        OUTPUT "High Food Waste"
```

```
    ELSE
        OUTPUT "Acceptable Food Waste"
    END IF
END FOR
STOP
```

7. Python Source Code

```
prepared = [100, 120, 80]
served = [90, 100, 60]

for i in range(len(prepared)):
    leftover = prepared[i] - served[i]
    percentage = leftover / prepared[i] * 100

    print("\nDay", i + 1)
    print("Prepared Meals:", prepared[i])
    print("Served Meals:", served[i])
    print("Leftover Meals:", leftover)
    print("Wastage Percentage:", round(percentage, 2))

    if percentage > 20:
        print("High Food Waste")
    else:
        print("Acceptable Food Waste")
```

8. Test Cases

Input / Scenario	Expected Result
Prepared 100, served 90	10 leftover; 10%; Acceptable
Prepared 120, served 100	20 leftover; 16.67%; Acceptable
Prepared 80, served 60	20 leftover; 25%; High Food Waste

9. Output Screenshot / Sample Output

This is a screenshot-style text capture of a representative run. Capture a real terminal screenshot after running the code in Python IDLE or VS Code if your faculty requires an actual image.

```
Day 1
Prepared Meals: 100
Served Meals: 90
Leftover Meals: 10
Wastage Percentage: 10.0
Acceptable Food Waste

Day 2
Prepared Meals: 120
Served Meals: 100
Leftover Meals: 20
Wastage Percentage: 16.67
Acceptable Food Waste

Day 3
Prepared Meals: 80
Served Meals: 60
Leftover Meals: 20
Wastage Percentage: 25.0
High Food Waste
```

10. Conclusion

The program calculates daily food waste and identifies high-waste days using lists, arithmetic, loops, and conditions.